

Unidad V: Árboles

Clase 8 – Pensar jerárquicamente: introducción a Árboles y ABB

Objetivo

Comprender estructuras jerárquicas y su potencia para búsqueda ordenada.

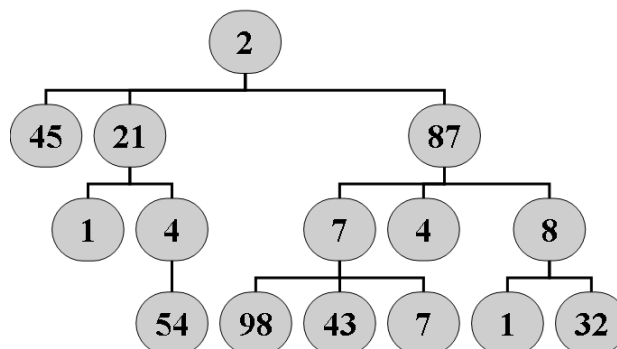
Tema

- Concepto de árbol.
- Recursividad.
- Árbol Binario de Búsqueda (ABB).
- Recorridos.

Concepto de árbol

Un **árbol** es una estructura de datos jerárquica formada por elementos llamados **nodos**. Entre esos nodos existe una relación de paternidad: un nodo puede ser padre de otros nodos, y esos otros nodos son sus hijos. El apunte define el árbol como una colección de nodos sobre la que se establece una relación de paternidad que impone una estructura jerárquica. En un árbol, todo nodo tiene un único padre, excepto un nodo especial llamado raíz, que no tiene padre.

Desde una mirada más formal, un árbol puede entenderse de manera recursiva: puede estar vacío, o puede estar formado por un nodo raíz y un conjunto de subárboles que dependen de esa raíz. Esta definición es importante porque anticipa cómo se resuelven muchos problemas con árboles: se procesa una raíz y luego se repite el mismo razonamiento sobre sus subárboles.

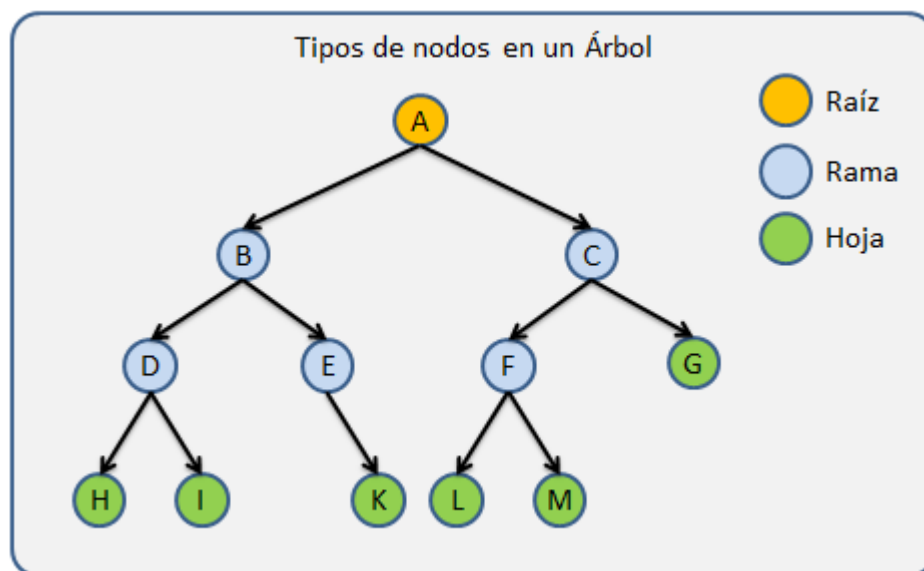


Los árboles aparecen en muchos contextos. Una estructura de carpetas en una computadora puede verse como un árbol: una carpeta principal contiene carpetas internas, y cada una de ellas puede contener otras carpetas. Un organigrama empresarial también puede representarse como un árbol: una dirección general tiene

gerencias, cada gerencia tiene jefaturas, y cada jefatura tiene equipos. Un árbol genealógico representa generaciones y relaciones familiares. En programación, los árboles permiten organizar información para facilitar búsquedas, clasificaciones y recorridos.

Componentes principales de un árbol

Para trabajar con árboles es necesario manejar una terminología básica. Estos conceptos no son accesorios: se usan constantemente para explicar algoritmos, recorridos, costos y problemas.



La **raíz** es el nodo inicial del árbol. Es el único nodo que no tiene padre. Desde la raíz se accede al resto de la estructura.

Un **nodo** es cada elemento almacenado en el árbol. En los ejemplos de la materia, los nodos suelen contener valores enteros, aunque en sistemas reales podrían almacenar objetos, registros, claves, referencias u otro tipo de información.

Una **rama** o **arista** es la conexión entre un nodo padre y un nodo hijo. En los dibujos suele representarse mediante una línea.

El **padre** de un nodo es el nodo que se encuentra inmediatamente por encima de él en la jerarquía.

Un **hijo** es un nodo que depende directamente de otro nodo.

Los **hermanos** son nodos que tienen el mismo padre.

Una **hoja** es un nodo que no tiene hijos. Es decir, es un punto final del árbol.

Un **nodo interno** es un nodo que no es hoja y que tiene al menos un hijo. En algunas definiciones se distingue además que los nodos internos no incluyen a la raíz si se la

quiere tratar como caso especial, pero en la práctica se suele llamar interno a todo nodo que tiene descendientes.

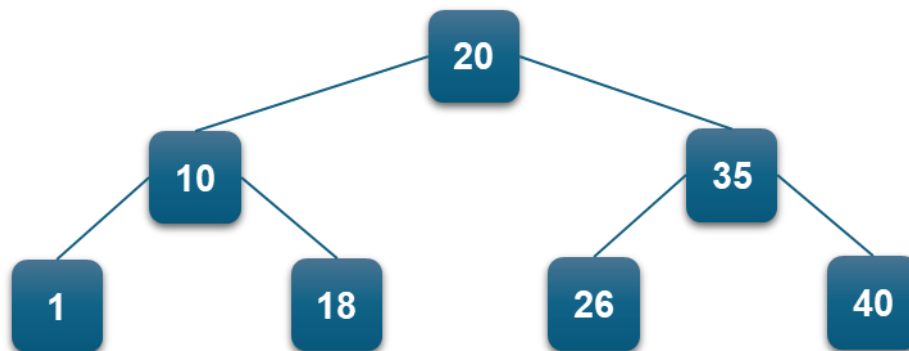
Los **descendientes** de un nodo son todos los nodos que dependen de él, directa o indirectamente.

Los **ancestros** de un nodo son su padre, el padre de su padre, y así sucesivamente hasta llegar a la raíz.

La **profundidad** de un nodo indica la distancia desde la raíz hasta ese nodo. Habitualmente, la raíz tiene profundidad 0. Sus hijos tienen profundidad 1, los hijos de sus hijos profundidad 2, y así sucesivamente.

La **altura** de un árbol es la mayor profundidad alcanzada por alguno de sus nodos. Dicho de otra forma, indica cuántos niveles de descenso tiene el árbol desde la raíz hasta la hoja más profunda. El apunte de cátedra utiliza estos conceptos para describir los árboles y ejemplifica profundidad, altura, hojas, descendientes y ancestros sobre un árbol de enteros.

Ejemplo



En este árbol, la raíz es 20. Las hojas son 1, 18, 26 y 40, porque no tienen hijos. Los hijos de 20 son 10 y 35. El padre de 18 es 10. Los nodos 10 y 35 son hermanos. La profundidad de 20 es 0, la profundidad de 10 y 35 es 1, y la profundidad de 1, 18, 26 y 40 es 2. La altura del árbol es 2.

TDA Árbol

Como en el resto de la materia, no alcanza con pensar una estructura por su dibujo. También debe pensarse como **Tipo de Dato Abstracto**, es decir, como un conjunto de operaciones que definen qué se puede hacer con esa estructura, independientemente de cómo se implemente internamente.

El **TDA Árbol** representa el comportamiento abstracto de un árbol. En el apunte, para definir el TDA de un árbol general se utilizan los conceptos de **hijo mayor** y **hermano siguiente**. El hijo mayor es el subárbol ubicado más a la izquierda. El hermano siguiente

es el subárbol que comparte el mismo padre y se encuentra a la derecha del nodo actual.

Esta forma de representación permite trabajar con árboles que no necesariamente son binarios. Por ejemplo, un nodo podría tener tres, cuatro o más hijos. En lugar de guardar una referencia a cada hijo posible, se guarda una referencia al primer hijo y una referencia al siguiente hermano. Esta idea será importante más adelante al estudiar árboles genéricos o n-arios.

Una posible especificación del TDA Árbol general, siguiendo el estilo del apunte, es:

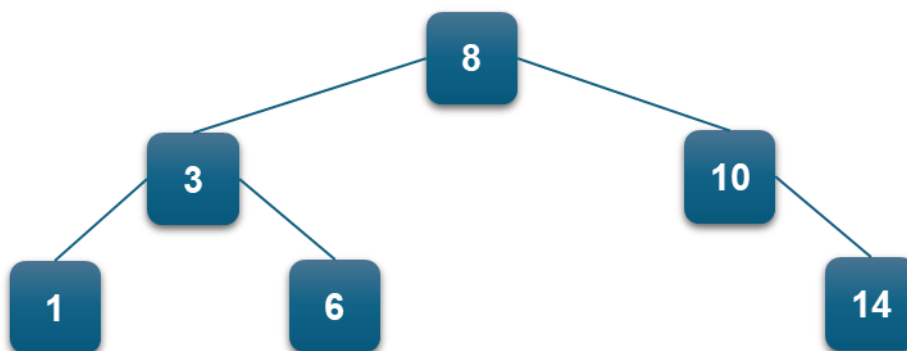
```
public interface ANaTDA {
    int Valor();
    ANaTDA HijoMayor();
    ANaTDA HermanoSig();
    boolean ArbolVacio();
    void InicializarArbol();
    void ElimHijoMConDesc(int x);
    void ElimHermSConDesc(int x);
    void CrearArbol(int r);
    void AgregarHijoM(int p, int h);
}
```

En esta clase, el TDA Árbol se introduce como marco conceptual. La implementación completa del árbol general no será el foco principal de la clase, porque el objetivo práctico inmediato será trabajar con árboles binarios y ABB. Sin embargo, es importante reconocer que ABB no es “el único árbol”, sino un caso particular dentro de una familia más amplia de estructuras jerárquicas.

Árbol binario

Un **árbol binario** es un caso particular de árbol en el que cada nodo puede tener como máximo dos hijos. Esos hijos se denominan **hijo izquierdo** e **hijo derecho**. Por lo tanto, un nodo de un árbol binario puede tener cero hijos, un hijo o dos hijos. El apunte presenta el árbol binario como un caso particular de árbol que cumple las definiciones generales, pero restringe la cantidad de hijos a lo sumo a dos.

Ejemplo de árbol binario:



Este árbol es binario porque ningún nodo tiene más de dos hijos. El nodo 8 tiene dos hijos: 3 y 10. El nodo 3 también tiene dos hijos: 1 y 6. El nodo 10 tiene solo un hijo derecho: 14. Los nodos 1, 6 y 14 son hojas.

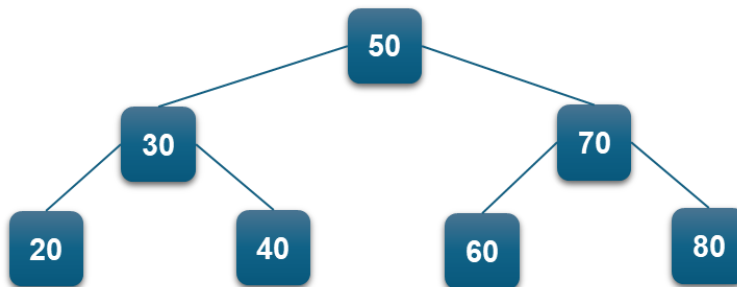
Es importante destacar que un árbol binario no necesariamente está ordenado. Que un árbol sea binario solo significa que cada nodo tiene como máximo dos hijos. Para que sea un árbol de búsqueda se necesita agregar una regla adicional.

Árbol Binario de Búsqueda: ABB

Un **Árbol Binario de Búsqueda**, abreviado **ABB**, es un árbol binario que incorpora una regla de orden. En inglés se lo conoce como **Binary Search Tree**, abreviado **BST**. En la materia usaremos la sigla ABB.

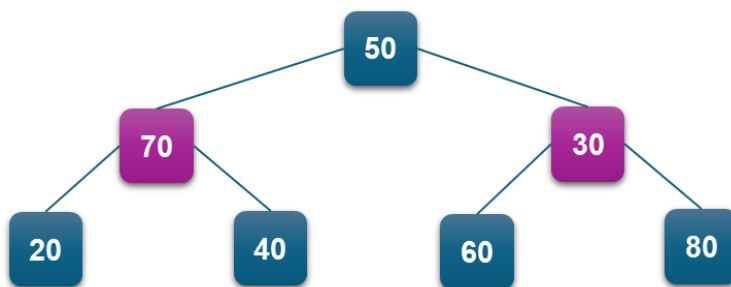
La regla es la siguiente: para cada nodo, todos los valores del subárbol izquierdo deben ser menores que el valor del nodo, y todos los valores del subárbol derecho deben ser mayores. Además, en la definición usada por el apunte, el ABB no admite elementos repetidos.

Ejemplo de ABB:



Este árbol es un ABB porque todos los valores a la izquierda de 50 son menores que 50, y todos los valores a la derecha son mayores. Lo mismo ocurre con cada nodo interno. A la izquierda de 30 está 20, que es menor; a la derecha está 40, que es mayor. A la izquierda de 70 está 60, que es menor; a la derecha está 80, que es mayor.

Ejemplo de árbol binario que no es ABB:



Este árbol es binario, porque cada nodo tiene como máximo dos hijos, pero no es ABB. El nodo 70 está a la izquierda de 50, aunque es mayor. El nodo 30 está a la derecha de 50, aunque es menor. Por lo tanto, no respeta la regla de búsqueda.

La potencia del ABB está en que permite decidir hacia dónde avanzar durante una búsqueda. Si se busca un valor menor que la raíz, no tiene sentido buscar en el subárbol derecho. Si se busca un valor mayor, no tiene sentido buscar en el subárbol izquierdo. Esta regla evita revisar todo el árbol cuando la estructura está bien distribuida.

Construcción manual de un ABB

Para construir un ABB se insertan los elementos de a uno, respetando siempre la regla de orden. El primer valor insertado se convierte en la raíz. Cada valor siguiente se compara con la raíz y se decide si debe ir a la izquierda o a la derecha. Si el lugar ya está ocupado, se continúa comparando con el nodo correspondiente hasta encontrar una posición vacía.

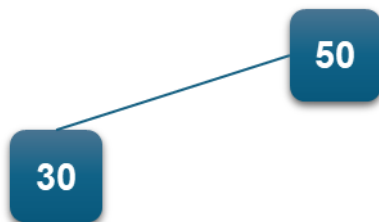
Supongamos la siguiente secuencia:

50, 30, 70, 20, 40, 60, 80

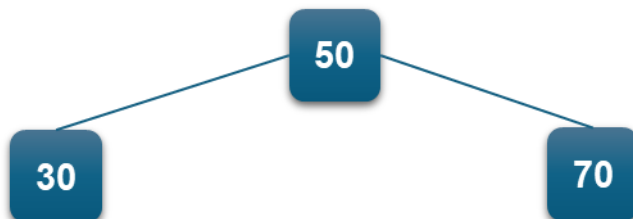
Primero se inserta 50, que será la raíz:



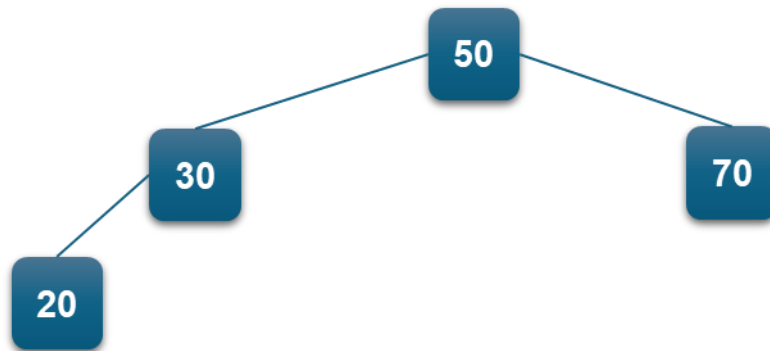
Luego se inserta 30. Como 30 es menor que 50, va a la izquierda:



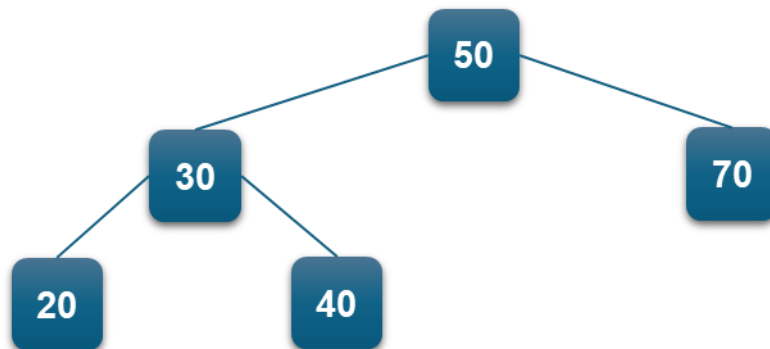
Luego se inserta 70. Como 70 es mayor que 50, va a la derecha:



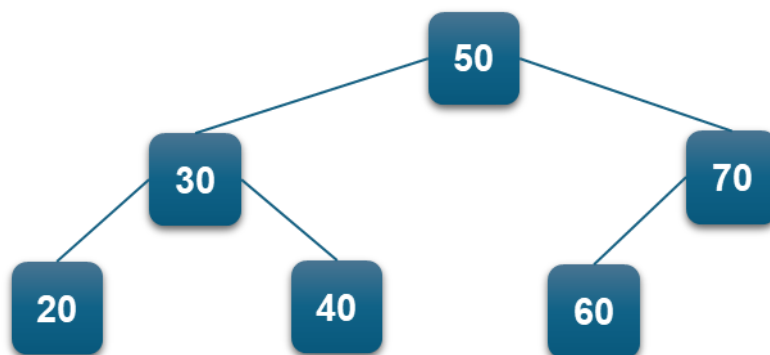
Luego se inserta 20. Se compara con 50: como es menor, se va a la izquierda. Se compara con 30: como es menor, se ubica a la izquierda de 30.



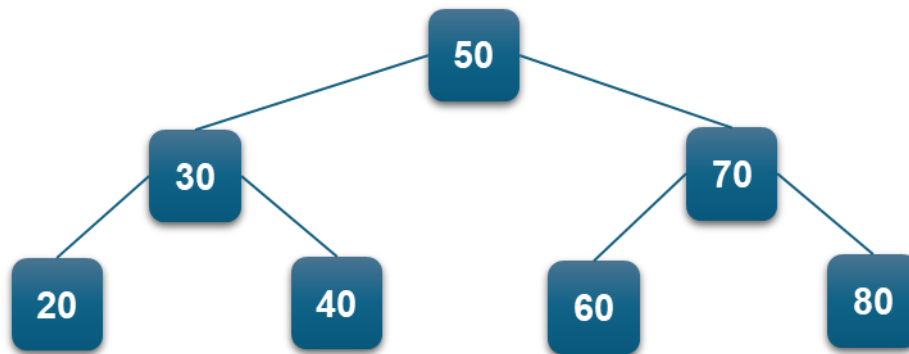
Luego se inserta 40. Es menor que 50, por lo tanto se baja a 30. Como 40 es mayor que 30, se ubica a la derecha de 30.



Luego se inserta 60. Es mayor que 50, se baja a 70. Como 60 es menor que 70, se ubica a la izquierda de 70.



Finalmente se inserta 80. Es mayor que 50, se baja a 70. Como 80 es mayor que 70, se ubica a la derecha de 70.



Esta actividad es fundamental porque permite visualizar cómo el ABB organiza la información. No se inserta según el orden de llegada físico, como en una cola, ni según el último ingresado, como en una pila. Se inserta según una regla de comparación.

Especificación del TDA ABB

El ABB también puede definirse como TDA. La especificación indica qué operaciones se pueden realizar, sin depender todavía de una implementación particular. Según el apunte, las operaciones centrales son: recuperar la raíz, obtener el hijo izquierdo, obtener el hijo derecho, verificar si el árbol está vacío, inicializar el árbol, agregar un elemento y eliminar un elemento manteniendo la propiedad de ABB.

Una especificación posible es:

```

public interface ABBTDA {
    // Precondición: el árbol está inicializado y no está vacío
    int Raiz();

    // Precondición: el árbol está inicializado y no está vacío
    ABBTDA HijoIzq();

    // Precondición: el árbol está inicializado y no está vacío
    ABBTDA HijoDer();

    // Precondición: el árbol está inicializado
    boolean ArbolVacio();

    // Inicializa el árbol
    void InicializarArbol();

    // Agrega un elemento manteniendo la propiedad de ABB
    void AgregarElem(int x);

    // Elimina un elemento manteniendo la propiedad de ABB
    void EliminarElem(int x);
}
  
```

En esta clase se priorizará la comprensión de Raiz, HijoIzq, HijoDer, ArbolVacio, InicializarArbol y AgregarElem. La operación EliminarElem se presentará como parte de la especificación, pero su desarrollo completo conviene trabajarlo más adelante, porque tiene varios casos y puede desviar el foco inicial.

Implementación básica del ABB

Una forma natural de implementar un ABB es mediante nodos dinámicos. Cada nodo almacena un valor y referencias a sus dos subárboles: el izquierdo y el derecho. El apunte presenta una implementación donde el nodo contiene un entero info y dos referencias de tipo ABBTDA, una para el hijo izquierdo y otra para el hijo derecho.

```
class NodoABB {
    int info;
    ABBTDA hijoIzq;
    ABBTDA hijoDer;
}
```

Luego, la clase ABB puede tener una referencia a la raíz:

```
public class ABB implements ABBTDA {
    NodoABB raiz;

    public void InicializarArbol() {
        raiz = null;
    }

    public boolean ArbolVacio() {
        return raiz == null;
    }

    public int Raiz() {
        return raiz.info;
    }

    public ABBTDA HijoIzq() {
        return raiz.hijoIzq;
    }

    public ABBTDA HijoDer() {
        return raiz.hijoDer;
    }
}
```

La lógica de inicialización es similar a otras estructuras dinámicas: si la raíz es null, el árbol está vacío. Cuando se agrega el primer elemento, se crea un nodo que pasa a ser la raíz. A diferencia de una lista, cada nodo no tiene un único sig, sino dos referencias: una hacia el subárbol izquierdo y otra hacia el subárbol derecho.

Agregar elementos en un ABB

La operación AgregarElem debe respetar la regla del ABB. Si el árbol está vacío, se crea la raíz. Si el árbol no está vacío, se compara el valor a insertar con la raíz. Si el valor es menor, se inserta en el subárbol izquierdo. Si es mayor, se inserta en el subárbol derecho. Si es igual, no se agrega porque el ABB no admite repetidos.

```
public void AgregarElem(int x) {
    if (raiz == null) {
        raiz = new NodoABB();
        raiz.info = x;
    }
}
```

```

    raiz.hijoIzq = new ABB();
    raiz.hijoIzq.InicializarArbol();

    raiz.hijoDer = new ABB();
    raiz.hijoDer.InicializarArbol();
} else if (x < raiz.info) {
    raiz.hijoIzq.AgregarElem(x);
} else if (x > raiz.info) {
    raiz.hijoDer.AgregarElem(x);
}
}

```

Este método es recursivo. No contiene un ciclo while ni un for; la repetición ocurre porque el método se invoca a sí mismo sobre el subárbol izquierdo o derecho. El caso base aparece cuando se llega a un árbol vacío: allí se crea el nuevo nodo. El paso recursivo aparece cuando todavía no se encontró la posición y se continúa bajando por izquierda o por derecha.

Recursividad

La **recursividad** es una técnica en la cual un método se define en función de sí mismo. En un método recursivo siempre deben existir dos partes: un **caso base**, también llamado condición de corte, y un **paso recursivo**, que es la llamada del método a sí mismo. El apunte remarca que sin una condición de corte, el método nunca terminaría su ejecución; por eso, cada llamada recursiva debe acercarse al caso base.

Ejemplo clásico:

```

public int factorial(int n) {
    if (n == 0) {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

```

En árboles, la recursividad aparece de manera más natural que en otros temas. Esto ocurre porque un árbol puede definirse como vacío o como una raíz con subárboles. Entonces, muchos problemas se resuelven siguiendo esta estructura:

Si el árbol está vacío:

hago algo simple

Si no está vacío:

proceso la raíz

resuelvo el problema en el subárbol izquierdo

resuelvo el problema en el subárbol derecho

Por ejemplo, para mostrar todos los elementos de un árbol, no se necesita recorrer un arreglo desde la posición 0 hasta la última. Se visita un nodo y luego se repite el proceso sobre sus subárboles.

Recorridos de árboles

Un **recorrido** es una forma de visitar todos los nodos de un árbol. En inglés se usa el término **traversal**, que significa recorrido o recorrido sistemático de una estructura. Los recorridos son fundamentales porque permiten procesar todos los elementos del árbol en un orden determinado.

Los tres recorridos clásicos son:

- **Pre-order** o preorden.
- **In-order** o inorden.
- **Post-order** o postorden.

El apunte indica que estos tres recorridos son recursivos por definición y que son la base para resolver muchos problemas con árboles.

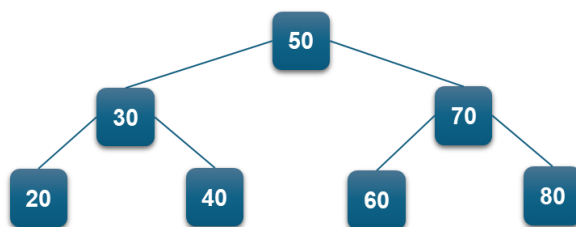
Recorrido pre-order o preorden

En pre-order se visita primero la raíz, luego el subárbol izquierdo y finalmente el subárbol derecho.

La regla es: Raíz - Izquierda - Derecha

Algoritmo:

```
public void preOrder(ABBTDA a) {
    if (!a.ArbolVacio()) {
        System.out.println(a.Raiz());
        preOrder(a.HijoIzq());
        preOrder(a.HijoDer());
    }
}
```



Recorrido pre-order: 50, 30, 20, 40, 70, 60, 80

Primero se visita la raíz 50. Luego se recorre el subárbol izquierdo de 50, comenzando por 30. Después se recorren sus hijos 20 y 40. Finalmente se recorre el subárbol derecho de 50, comenzando por 70, y luego sus hijos 60 y 80.

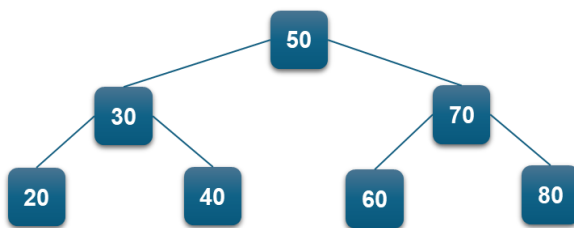
Recorrido in-order o inorden

En in-order se visita primero el subárbol izquierdo, luego la raíz y finalmente el subárbol derecho.

La regla es: Izquierda - Raíz - Derecha

Algoritmo:

```
public void inOrder(ABBTDA a) {
    if (!a.ArbolVacio()) {
        inOrder(a.HijoIzq());
        System.out.println(a.Raiz());
        inOrder(a.HijoDer());
    }
}
```



Recorrido in-order: 20, 30, 40, 50, 60, 70, 80

Este recorrido es especialmente importante porque, en un ABB, devuelve los valores ordenados de menor a mayor. Esto sucede porque primero se visita todo lo menor, luego la raíz, y finalmente todo lo mayor.

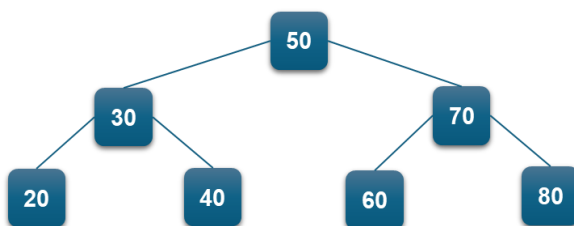
Recorrido post-order o postorden

En post-order se visitan primero los subárboles y al final la raíz.

La regla es: Izquierda - Derecha - Raíz

Algoritmo:

```
public void postOrder(ABBTDA a) {
    if (!a.ArbolVacio()) {
        postOrder(a.HijoIzq());
        postOrder(a.HijoDer());
        System.out.println(a.Raiz());
    }
}
```

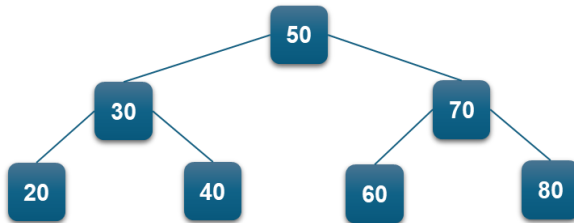


Recorrido post-order: 20, 40, 30, 60, 80, 70, 50

Este recorrido procesa primero los hijos y deja la raíz para el final. Es útil cuando se necesita resolver primero las partes inferiores de la estructura antes de procesar el nodo principal.

Comparación de recorridos

Sobre el mismo árbol:



Los recorridos son:

Pre-order: 50, 30, 20, 40, 70, 60, 80

In-order: 20, 30, 40, 50, 60, 70, 80

Post-order: 20, 40, 30, 60, 80, 70, 50

La diferencia está en el momento en que se visita la raíz. En pre-order, la raíz se visita antes que sus hijos. En in-order, la raíz queda entre el subárbol izquierdo y el derecho. En post-order, la raíz se visita después de sus hijos.

Para recordar los recorridos, conviene mirar dónde aparece la raíz:

- Pre-order: raíz primero.
- In-order: raíz en el medio.
- Post-order: raíz al final.

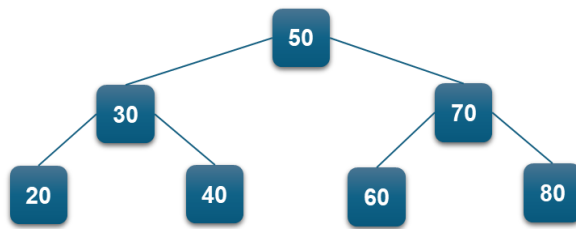
Búsqueda en un ABB

Buscar un elemento en un ABB consiste en comparar el valor buscado con la raíz. Si coincide, se encontró. Si el valor buscado es menor, se continúa por el subárbol izquierdo. Si es mayor, se continúa por el subárbol derecho.

```

public boolean existe(ABBTDA a, int x) {
    if (a.ArbolVacio()) {
        return false;
    } else if (a.Raiz() == x) {
        return true;
    } else if (x < a.Raiz()) {
        return existe(a.HijoIzq(), x);
    } else {
        return existe(a.HijoDer(), x);
    }
}
  
```

En este método, el caso base aparece cuando el árbol está vacío o cuando se encuentra el valor. El paso recursivo ocurre cuando se decide continuar la búsqueda por izquierda o por derecha.

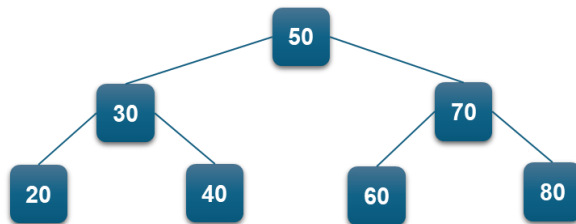


Ejemplo: buscar 60 en el siguiente ABB: Primero se compara 60 con 50. Como 60 es mayor, se baja a la derecha. Luego se compara 60 con 70. Como 60 es menor, se baja a la izquierda. Finalmente se llega a 60, por lo tanto el valor existe.

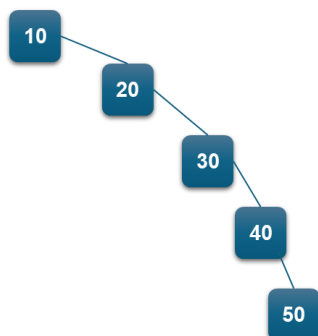
Una primera mirada al costo

El costo de búsqueda en un ABB depende de la forma del árbol. Si el árbol está relativamente equilibrado, cada comparación permite descartar una parte importante de la estructura, y la búsqueda se acerca a un comportamiento logarítmico. Si el árbol está totalmente desbalanceado, puede parecerse a una lista, y en ese caso la búsqueda puede requerir recorrer muchos nodos.

Ejemplo de ABB equilibrado:



Ejemplo de ABB desbalanceado:



Ambos son ABB, pero no tienen el mismo rendimiento. En el primero, la búsqueda baja pocos niveles. En el segundo, buscar 50 implica pasar por todos los nodos anteriores. Este problema será retomado en profundidad cuando se estudien altura, balance conceptual y árboles AVL.